

OBD2 PORT READER

Hunter Pinkstaff

Bryan Lew

Mark Negus

EE329-07

Spring 2025

FINAL PROJECT

June 6, 2025

Penvenne

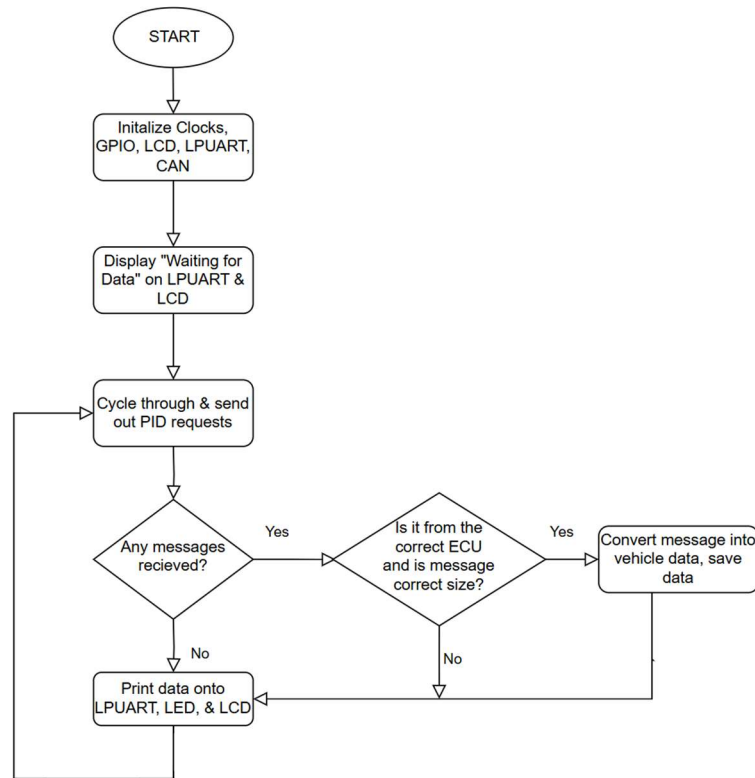


Figure 1. High-Level Flowchart

Behavior Description:

The OBD2 port reader is able to send and receive classic CAN PIDs through the car's CAN bus. It can then translate the received data into real-world values. It will present RPM, Speed, and coolant temperature on the LCD. It can also display the current rpm on a display composed of six LEDs, along with an added LED to indicate when to shift to the next gear. All additional data from CAN will be displayed on a virtual terminal through LPUART. The system is fully operational.

System Specifications:

Table 1. System specifications

Parameter	Value	Units	Comments:
STM32 Clock	80	MHz	Applies to both STM32s
CAN BUS Baud Rate	500	kbps	
LPUART Baud Rate	115.2	kbps	
LCD Driver interface	4	bits	8-bit data will be sent in 2 4-bit packages
STM32 Supply voltage	5	V	In-car: Supplied by voltage regulator Simulation: Supplied by ST-Link

CAN Transceiver supply voltage	5	V	In-car: Supplied by voltage regulator Simulation: Supplied by Nucleo
LED/LCD Supply voltage	3.3	V	Supplied by Nucleo
LED Power Consumption	85.75	mW	
Total System Current	252	mA	Measured at the CAN transceiver
Total System Size	22x14.6x5	cm ³	Size of all components
OBD2 cable length	62	cm	

Code Planning & Software Architecture:

The main.c file primarily deals with LCD printing, hex PID math, and calling the “request” and “receive” functions from CAN1.c. The infinite while loop constantly cycles through and sends out an array of PID requests. Then it checks the messages received and will only read the data if the message is a single frame, coming from a desired ECU, and containing a return tag (0x41). It will then match the PID to its corresponding function and manipulate the raw data into data that can be displayed and stores it as a variable. After this, the data is displayed on LPUART, the LCD, & LEDs.

CAN1.c is the bulk of our code. It first initializes the CAN bit rate, registers, and peripherals. The initialization function will set the CAN clock to 80MHz, make the initialization request flag high, enable automatic bus management, configure the bit timing register and filter master register, and then exit initialization mode. The BTR is important for timing as it includes synchronization bits and buffers. The BTR has sampling occur at $11 + 1 = 12/16$ time quanta or 75% through the bit. This was a very subtle detail learned during the initialization process. This was done since it is recommended to sample at 75-90% of a CAN bit for stability purposes. The steps on how the send and receive functions work are listed below:

General recipe for sending/receiving

- Wait for the controller’s “message pending” (or empty mailbox if you are sending a frame) flag
- Read or specify the header word: identifier, frame type, and length
- Load in the high and low data bytes (4 each)
- Tell the controller you’re done with the slot so it can queue the next frame

Pseudo Code

Below is the pseudocode for Main.c and CAN1.c (LPUART, LCD, etc were unchanged)

Main.c

START OBD2_Reader

----- GLOBAL DATA -----

rpm -> 32-bit **unsigned**
coolant_temp -> 16-bit **unsigned**
speed_kph -> 8-bit **unsigned**
...more values as needed...

PIDs_to_request -> { 0x0C , 0x05 , 0x0D , ... }
pid_idx -> 0

----- MAIN FLOW -----

- Power-up initialisation
 - bring up HAL, clock, GPIO, shift-lights, LCD, UART, CAN @ 500 kbps
- Splash-screen
 - UART "TURN ON CAR TO BEGIN"
 - LCD line 1 "OBD2 Reader Rdy"
 - line 2 "Waiting data..."
- Forever loop
 - pid -> PIDs_to_request[pid_idx] // choose next PID
 - send PID request frame // see "PID request"
 - pid_idx -> (pid_idx + 1) mod total_PIDs
 - check CAN FIFO
 - if a single-frame response is waiting
 - > feed it to "Parse OBD2 frame" to refresh rpm / temp / speed / ...
 - refresh outputs
 - UART print latest values in tidy lines
 - LCD line 1 RPM, line 2 Coolant-temp and Speed

END LOOP

----- PID REQUEST -----
8-byte payload -> [02 01 pid 00 00 00 00 00]
transmit with CAN ID 0x7DF

----- PARSE OBD2 FRAME -----
accept frame only if
CAN ID in 0x7E8...0x7EF AND second byte == 0x41

A -> third data byte
B -> fourth data byte
VALUE -> (A << 8) + B // 16-bit combo

switch on PID (third byte)
0x0C -> rpm = VALUE / 4
0x05 -> coolant_temp = A - 40
0x0D -> speed_kph = A
0x04 -> engine_load = A ×100 / 255
0x5C -> oil_temp = A - 40
other -> ignore

----- END -----

CAN1.C

START CAN1

----- CAN1 INITIALIZATION @ 500 kbps -----

- Turn on CAN1 peripheral clock
- Request initialization mode -> wait until controller confirms
- Operating flags
 - exit sleep mode
 - enable automatic bus-off
 - enable automatic wake-up
 - enable automatic retransmission
- Bit timing -> prescaler, TS1, TS2, SJW chosen for 80 MHz ÷ 16 TQ = 500 kbps
- Normal operation -> loopback OFF, silent mode OFF
- Filter block
 - enter filter-setup mode

- filter 0 -> mask mode, 32-bit scale, FIFO 0
- ID = 0x00000000, mask = 0x00000000 (accept everything)
- activate filter 0 -> leave filter-setup mode
- Leave initialization-> CAN1 ready **for** traffic

----- TRANSMIT FRAME -----

Input id , data[] , length

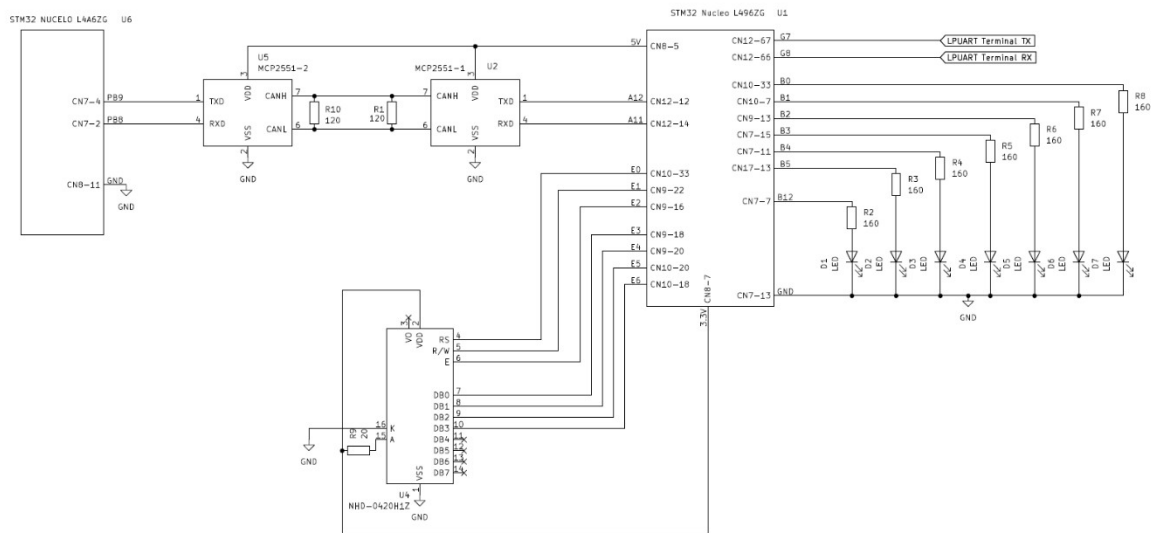
- **if** length > 8 -> **return** ERROR
- pick first free mailbox
- mailbox_num -> 0 **if** empty
 - 1 **else if** empty
 - 2 **else if** empty
 - error **if** none free
- load mailbox
 - identifier field -> id shifted into position
 - DLC field -> length
 - first 0-4 bytes -> low **register**
 - bytes 4-7 (**if** any) -> high **register**
- set transmit-request bit
- **return** SUCCESS

----- RECEIVE FRAME -----

Output id_out , data_out[] , length_out

- **if** FIFO-0 empty -> **return** NO_MESSAGE
- read from FIFO-0
 - id_out -> identifier bits
 - length_out -> DLC
 - data_out[] -> up to 8 bytes from low + high registers
- release FIFO-0 so next frame can queue
- **return** MESSAGE_RECEIVED

END CAN1



Calculation & Characterization

Theoretical LED Power Calculation: $7 * \frac{(3.3-2)^2}{160} = 73.94 \text{ mW}$

Measured LED Power Calculation: $7 * \frac{(3.3-1.9)^2}{160} = 85.75 \text{ mW}$

Measured LED Current: $7 * \frac{3.3-1.9}{160} = 61.25 \text{ mA}$

Total Current Draw (Measured through Voltage Regulator): 252mA

Total Measured Power Draw (5V)(252mA) = 1.26W

Attributions:

Hunter Pinkstaff

- Wrote the Main.c file for the project
- Crimped OBD II adapter wire
- Made OBD2-STM interface
- Created wiring schematics & broad flowchart

Mark Negus

- Wrote CAN1.c file
- Pseudocode for main.c/can1.c
- Debugged hardware – LEDs, simulated ECU

Bryan Lew

- Initialized peripheral source files
- Wrote and debugged the simulated CAR ECU STM32 code
- Integrated hardware for the car and simulated ECU STM32
- Wrote Bill of Materials
- Formatted and placed code in report
- Added references

Appendix A: Bill of Materials

Quantity	Part Type	Designator	Description	Part Number	Vendor	Unit Cost	Line Total
1	Voltage Regulator	U3	Takes ~12V battery power from car's OBD-II port & converts to 5V to power components	L7805CV	Radio-shack	5	5
2	CAN Transceiver (with Board)	U2,U5	Turns CAN RX/TX into CAN High/Low differential pair to send along CAN bus	MCP2551	Amazon	7.6	15.2
7	160Ω Resistors	R2,R3,R4, R5,R6,R7, R8	Current limiting resistors in series with RPM Indicator LEDs connected to 3.3V GPIO	N/A	Student Project Lab	0	0
2	120Ω Resistors	R1, R10	Termination resistors for CAN protocol to remove transmission-line effects/reflections	N/A	Student Project Lab	0	0
1	LCD	U4	16x2 LCD to display engine rpm, coolant temperature, and car speed in km/h	NHD-0420H1Z	Mouser	12.78	12.78
1	STM32 MCU Development Board	U1	Requests CAN data and outputs data to external peripherals	NUCLEO-L496ZG	Digikey	21.02	21.02
1	STM32 MCU Development Board	U6	Receives/Sends CAN messages to OBD2 Reader, acts as a virtual CAN bus	NUCLEO-L4A6ZG	Digikey	19.99	19.99
1	Jumper Wire Pack	N/A	Electrical Connections	110990029	Mouser	7.5	7.5

2	Breadboard	N/A	Board to contain all electrical connections	Breadboard	Student Project Lab	0	0
1	Cigar Box	N/A	Acts as housing for STM32 & breadboard to make it portable	N/A	F&M Smokes	0	0
1	Cable	N/A	Micro-USB to USB-A Cable (6 ft, 2-pack)	680360118548	Amazon	5.49	5.49
1	0.33 μ F Capacitor	C1	Voltage Regulator Input Filtering Capacitor	N/A	Student Project Lab	0	0
1	0.1 μ F Capacitor	C2	Voltage Regulator Output Filtering Capacitor	N/A	Student Project Lab	0	0
2	Green LEDs	D1,D2	Used to display lower rpm range	N/A	Student Project Lab	0	0
2	Yellow LEDs	D3,D4	Used to display middle rpm range	N/A	Student Project Lab	0	0
2	Red LEDs	D5,D6	Used to display upper rpm range	N/A	Student Project Lab	0	0
1	Blue LED	D7	Used to indicate when to shift gears	N/A	Student Project Lab	0	0
----	----	----	----	----	----	Sub-total	86.98
----	----	----	----	----	7.25% ====>	Sales Tax	6.31
----	----	----	----	----	----	Total	93.29

Appendix B: References

- [1] Arm Keil, “CAN primer: creating your own network—hands-on using the Keil Simulator or the STM32F4 Discovery Board,” Application Note 236, 2012. Accessed: May 10, 2025. [Online]. Available: https://www.keil.com/appnotes/files/apnt_236.pdf
- [2] Microchip Technology Inc., “MCP2551 – high-speed CAN transceiver,” Product page, 2016. Accessed: May 10, 2025. [Online]. Available: <https://www.microchip.com/en-us/product/mcp2551>
- [3] Y. Xie, X. Su, Y. He, and X. Chen, “STM32-based vehicle data acquisition system for Internet-of-Vehicles,” in *Proc. 2017 IEEE/ACIS 16th Int. Conf. Computer and Information Science (ICIS)*, Wuhan, China, May 2017, doi: 10.1109/ICIS.2017.7960119.
- [4] CSS Electronics, “OBD2 explained – a simple intro [Video],” YouTube, 2019. Accessed: May 10, 2025. [Online]. Available: https://www.youtube.com/watch?v=OhShoU_E-0g
- [5] CANable Community, “Getting started with CANable,” 2022. Accessed: May 25, 2025. [Online]. Available: <https://canable.io/getting-started.html>
- [6] FlexiHub, “OBD2 pinout explained: major car-brand pinouts,” 2023. Accessed: May 10, 2025. [Online]. Available: <https://www.flexihub.com/oobd2-pinout/>
- [7] CSS Electronics, “OBD2 PID overview: lookup/converter tool, table, CSV, DBC,” 2024. Accessed: May 10, 2025. [Online]. Available: <https://www.csselectronics.com/pages/obd2-pid-table-on-board-diagnostics-j1979>
- [8] YouTube, “Video ID CMGYUUAeSqU [Video],” 2024. Accessed: Jun. 5, 2025. [Online]. Available: <https://www.youtube.com/watch?v=CMGYUUAeSqU>
- [9] J. Mast, “Nucleo-L496ZG color-coded CN pinout,” Mbed platform page, 2025. Accessed: May 29, 2025. [Online]. Available: <https://os.mbed.com/platforms/ST-Nucleo-L496ZG/>
- [10] YouTube, “Video ID Z25tbyHpQuM [Video],” 2025. Accessed: Jun. 5, 2025. [Online]. Available: <https://www.youtube.com/watch?v=Z25tbyHpQuM>

Appendix C : Source code quality

SCANNER STM32

*no main.h

main.c

```
/*USER CODE BEGIN Header */
/* =====
 * EE 329 OBD2 Reader and Display
 * =====
 * @file    : main.c
 * @brief    : Main program for OBD2 data acquisition, processing,
 *              and display via LPUART and LCD.
 * Project   : EE329 S'25 OBD2 Project
 * Author:    Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * Version   : 1.0
 * Date      : 5/31/2025
 * compiler  : STM32CubeIDE v1.18.0
 * target    : NUCLEO-L496ZG
 * clocks    : 80Mhz HSI to PLL
 * @attention: (c) 2025 STMicroelectronics. All rights reserved
 *
 * Pin Assignments (ST Nucleo L496ZG):
 *
 * Function      | Pin Name | STM32-L496ZG Pin |
 * -----|-----|-----|
 * STM32 BOARD   |          |                   |
 * -----|-----|-----|
 * CAN RX        | PA11     | CN7-26   AF (CAN1) |
 * CAN TX        | PA12     | CN7-27   AF (CAN1) |
 * User Button (B1) | PC13    | CN11-23  IN         |
 * Relay Control  | PB5      | CN12-16  OUT         |
 * LPUART1 TX     | PC1      | CN11-36  AF (ST-Link VCP) |
 * LPUART1 RX     | PC0      | CN11-38  AF (ST-Link VCP) |
 * -----|-----|-----|
 * MAIN.C FUNCTIONS
 * =====
 * -Initializes system, CAN, LPUART, and LCD.
 * -Cycles through requesting various OBD2 PIDs
 * -Parses CAN messages and stores PID data in variables.
 * -Periodically updates LCD with key OBD2 data (RPM, Temp, Speed).
 * -Periodically updates LPUART with a list of all collected OBD2 data.
 *
 * PLAN
 * =====
 * 1. Test peripherals individually from other source files
 * 2. Attempt to run all peripherals simultaneously without errors
```

```

* 3. Implement CAN in init and while loop to try reading data from car
* 4. Tweak PID requests
*
* REVISION HISTORY
* =====
* 1. Reorder initializations
* 2. Tweaked PID and CAN handling in while loop
* 3. Changed PID values
*/
/* 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-2345678 */
/*USER CODE END HEADER */
/* Includes ----- */
#include "main.h"
#include "CAN1.h"
#include "LCD.h"
#include "delay.h"
#include "LPUART.h"
#include "LED.h"
#include <stdint.h>
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h> // For rand()
void BOARD_INIT(void);
void pid_request(uint8_t pid);
void obd_data_handler(uint32_t id, const uint8_t *d);
void LPUART_Display_Data(void);
void LCD_Display_Data(void);
void SystemClock_Config(void);
void Error_Handler(void);
uint32_t rpm = 0;
uint16_t coolant_temp = 0;
uint8_t speed_kph = 0;
uint16_t engine_load = 0;
uint16_t fuel_pressure = 0;
uint16_t intake_man_pressure = 0;
uint16_t intake_air_temp = 0;
uint32_t mass_airflow = 0;
uint16_t throttle_position = 0;
uint32_t car_run_time = 0;
uint32_t error_code_milage = 0;
uint16_t fuel_level = 0;
uint16_t CM_voltage = 0;
uint16_t outside_air_temp = 0;
uint16_t throttle_pos_2 = 0;
uint16_t accelerator_position_1 = 0;
uint16_t accelerator_position_2 = 0;
uint16_t oil_temp = 0;
const uint8_t PIDS_TO_REQUEST[] = { 0x0C, 0x05, 0x0D, 0x04, 0x0A, 0x0B, 0x0F,
    0x10, 0x11, 0x1F, 0x21, 0x2F, 0x42, 0x46, 0x47, 0x49, 0x4A, 0x5C };
const uint8_t NUM_PIDS_TO_REQUEST = sizeof(PIDS_TO_REQUEST)
    / sizeof(PIDS_TO_REQUEST[0]);
uint8_t PID_NUM_IDX = 0;
/*-----
* function : main()
* author   : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu

```

```

* input(s) : none
* output(s): int
* action   : Initializes system, CAN, LPUART, LCD, and manages
*           : data acquisition and display.
* version  : 1.0
* date     : 06/01/25
*-----*/
int main(void) {
    HAL_Init();
    SystemClock_Config();
    BOARD_INIT();
    ShiftLight_Init();
    LCD_init();
    LCD_start_msg();
    LPUART_init();
    LPUART_Print("TURN ON CAR TO BEGIN\r\n");
    CAN1_Init_500k();
    // Initial LCD Display
    LCD_command(0x01);
    delay_us(2000);
    LCD_command(0x80);
    LCD_write_string("OBD2 Reader Rdy");
    delay_us(2000);
    LCD_command(0xC0);
    LCD_write_string("Waiting data...");
    LPUART_Print("Waiting for data...\r\n");
    while (1) {
        // Cycle through sending OBD2 PID requests
        pid_request(PIDS_TO_REQUEST[PID_NUM_IDX]);
        PID_NUM_IDX = (PID_NUM_IDX + 1) % NUM_PIDS_TO_REQUEST;
        delay_us(2000);
        // Check for and process received CAN messages
        uint32_t id;
        uint8_t d[8], len;
        if (CAN1_Receive(&id, d, &len)) {
            if ((d[0] >> 4) == 0) { // Single frame
                obd_data_handler(id, d);
            }
        }
        // Update all displays
        delay_us(200);
        LPUART_Display_Data();
        LCD_Display_Data();
        DisplayRPM(rpm);
    }
}

/*-----
* function : BOARD_INIT()
* author   : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : none
* output(s): none
* action   : Configures GPIO pins for CAN communication (PA11, PA12)
* version  : 1.0
* date     : 06/01/25

```

```

/*-----*/
void BOARD_INIT(void) {
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
    GPIOA->MODER &= ~(GPIO_MODER_MODE11_Msk | GPIO_MODER_MODE12_Msk);
    GPIOA->MODER |= (GPIO_MODER_MODE11_1 | GPIO_MODER_MODE12_1);
    GPIOA->AFR[1] &= ~((0xF << GPIO_AFRH_AFSEL11_Pos)
        | (0xF << GPIO_AFRH_AFSEL12_Pos));
    GPIOA->AFR[1] |= ((0x9 << GPIO_AFRH_AFSEL11_Pos)
        | (0x9 << GPIO_AFRH_AFSEL12_Pos)); // AF9
    GPIOA->OSPEEDR |= ((3U << GPIO_OSPEEDR_OSPEED11_Pos)
        | (3U << GPIO_OSPEEDR_OSPEED12_Pos));
}

/*-----*/
* function : pid_request()
* author   : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : pid (uint8_t)
* output(s): none
* action   : Sends a standard OBD-II request for a specific PID.
* version  : 1.0
* date     : 06/01/25
/*-----*/
void pid_request(uint8_t pid) {
    uint8_t msg[8] = { 0x02, 0x01, pid, 0x00, 0x00, 0x00, 0x00, 0x00 };
    CAN1_Send(0x7DF, msg, 8);
}

/*-----*/
* function : obd_data_handler()
* author   : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : id (uint32_t), *d (uint8_t)
* output(s): none
* action   : Parses received OBD-II single-frame CAN messages & updates data
* version  : 1.0
* date     : 06/01/25
/*-----*/
void obd_data_handler(uint32_t id, const uint8_t *d) {
    if ((id < 0x7E8 || id > 0x7EF) || (d[1] != 0x41))
        return; //checks if data is classic can & return data
    //parse data in prep for calculation
    uint8_t A = d[3];
    uint8_t B = d[4];
    uint16_t AB = (A << 8) | B;
    switch (d[2]) //checking PID of obd 2 responses
    {
        case 0x0C: {
            rpm = (AB + 2) / 4;
            break;
        }
        case 0x05: {
            coolant_temp = A - 40;
            break;
        }
        case 0x0D: {
            speed_kph = A;
        }
    }
}

```

```

    break;
}
case 0x04: {
    engine_load = A * 100 / 255;
    break;
}
case 0x0A: {
    fuel_pressure = A * 3;
    break;
}
case 0x0B: {
    intake_man_pressure = A;
    break;
}
case 0x0F: {
    intake_air_temp = A - 40;
    break;
}
case 0x10: {
    mass_airflow = AB;
    break;
}
case 0x11: {
    throttle_position = A * 100 / 255;
    break;
}
case 0x1F: {
    car_run_time = AB;
    break;
}
case 0x21: {
    error_code_milage = AB;
    break;
}
case 0x2F: {
    fuel_level = A * 100 / 255;
    break;
}
case 0x42: {
    CM_voltage = AB;
    break;
}
case 0x46: {
    outside_air_temp = A - 40;
    break;
}
case 0x47: {
    throttle_pos_2 = A * 100 / 255;
    break;
}
case 0x49: {
    accelerator_position_1 = A * 100 / 255;
    break;
}
case 0x4A: {

```

```

        accelerator_position_2 = A * 100 / 255;
        break;
    }
    case 0x5C: {
        oil_temp = A - 40;
        break;
    }
    default: {
        char buf[30];
        snprintf(buf, sizeof(buf), "Unknown PID: 0x%02X\r\n", d[2]);
        LPUART_Print(buf);
        break;
    }
}
}

/*-----*/
* function : LPUART_Display_Data()
* author   : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : none
* output(s): none
* action   : Clears LPUART screen & displays all collected OBD-II data values
* version  : 1.0
* date     : 06/01/25
*-----*/
void LPUART_Display_Data(void) {
    char buf[50];
    LPUART_ESC_Print("[2J\033[H");
    LPUART_Print("--- OBD-II Data ---\r\n");
    LPUART_Print("-----\r\n");
    snprintf(buf, sizeof(buf), "RPM: %lu\r\n", rpm);
    LPUART_Print(buf);
    if (coolant_temp != 0) {
        snprintf(buf, sizeof(buf), "Coolant Temp: %d C\r\n", coolant_temp);
    } else {
        snprintf(buf, sizeof(buf), "Coolant Temp: ---\r\n");
    }
    LPUART_Print(buf);
    snprintf(buf, sizeof(buf), "Speed: %u km/h\r\n", speed_kph);
    LPUART_Print(buf);
    snprintf(buf, sizeof(buf), "Engine Load: %u.%02u %%\r\n", engine_load / 100,
        engine_load % 100);
    LPUART_Print(buf);
    snprintf(buf, sizeof(buf), "Fuel Press: %u kPa\r\n", fuel_pressure);
    LPUART_Print(buf);
    snprintf(buf, sizeof(buf), "Intake Manifold Pressure: %u kPa\r\n",
        intake_man_pressure);
    LPUART_Print(buf);
    if (intake_air_temp != 0) {
        snprintf(buf, sizeof(buf), "Intake Air Temp: %d C\r\n", intake_air_temp);
    } else {
        snprintf(buf, sizeof(buf), "Intake Air Temp: ---\r\n");
    }
    LPUART_Print(buf);
    snprintf(buf, sizeof(buf), "Mass Airflow: %lu.%02lu g/s\r\n",

```

```

        mass_airflow / 100, mass_airflow % 100);
LPUART_Print(buf);
snprintf(buf, sizeof(buf), "Throttle Pos: %u.%02u %%\r\n",
        throttle_position / 100, throttle_position % 100);
LPUART_Print(buf);
snprintf(buf, sizeof(buf), "Run Time: %lu s\r\n", car_run_time);
LPUART_Print(buf);
snprintf(buf, sizeof(buf), "Error Code Milage: %lu km\r\n",
        error_code_milage);
LPUART_Print(buf);
snprintf(buf, sizeof(buf), "Fuel Level: %u.%02u %%\r\n", fuel_level / 100,
        fuel_level % 100);
LPUART_Print(buf);
snprintf(buf, sizeof(buf), "Car Battery Voltage: %u.%03u V\r\n",
        CM_voltage / 1000, CM_voltage % 1000);
LPUART_Print(buf);
if (outside_air_temp != 0) {
    snprintf(buf, sizeof(buf), "Ambient Air Temp: %d C\r\n",
        outside_air_temp);
} else {
    snprintf(buf, sizeof(buf), "Ambient Air Temp: ---\r\n");
}
LPUART_Print(buf);
snprintf(buf, sizeof(buf), "Abs Throt B: %u.%02u %%\r\n",
        throttle_pos_2 / 100, throttle_pos_2 % 100);
LPUART_Print(buf);
snprintf(buf, sizeof(buf), "Accel Ped D: %u.%02u %%\r\n",
        accelerator_position_1 / 100, accelerator_position_1 % 100);
LPUART_Print(buf);
snprintf(buf, sizeof(buf), "Accel Ped E: %u.%02u %%\r\n",
        accelerator_position_2 / 100, accelerator_position_2 % 100);
LPUART_Print(buf);
if (oil_temp != 0) {
    snprintf(buf, sizeof(buf), "Engine Oil Temp: %d C\r\n", oil_temp);
} else {
    snprintf(buf, sizeof(buf), "Engine Oil Temp: ---\r\n");
}
LPUART_Print(buf);
snprintf(buf, sizeof(buf), "Req PID: 0x%02X\r\n",
        PIDS_TO_REQUEST[PID_NUM_IDX]);
LPUART_Print(buf);
LPUART_Print("-----\r\n");
}

/*-----
 * function : LCD_Display_Data()
 * author   : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * input(s) : none
 * output(s): none
 * action   : Displays RPM, Coolant Temp, and Speed on the LCD
 * version  : 1.0
 * date     : 06/01/25
 *-----*/
void LCD_Display_Data(void) {
    char lcd_buf[17];

```

```

    sprintf(lcd_buf, "RPM: %-5lu      ", rpm);
    lcd_buf[16] = '\0';
    LCD_command(0x80);
    LCD_write_string(lcd_buf);
    char temp_str[5];
    char kph_str[4];
    if (coolant_temp == 0) {
        sprintf(temp_str, "---F");
    } else {
        sprintf(temp_str, "%dF", coolant_temp);
    }
    if (speed_kph == 0 && rpm == 0) {
        sprintf(kph_str, "---");
    } else {
        sprintf(kph_str, "%u", speed_kph);
    }
    snprintf(lcd_buf, sizeof(lcd_buf), "T:%-4s S:%-3skmh", temp_str, kph_str);
    int current_len = 0;
    while (lcd_buf[current_len] != '\0' && current_len < 16)
        current_len++;
    for (int i = current_len; i < 16; ++i) {
        lcd_buf[i] = ' ';
    }
    lcd_buf[16] = '\0';
    LCD_command(0xC0);
    LCD_write_string(lcd_buf);
}

/* -----
 * function : SystemClock_Config
 * action   : Configures the microcontroller's system clock to 80MHz.
 * author   : STM32 Authors
 * ----- */
void SystemClock_Config(void) {
    RCC_OscInitTypeDef RCC_OscInitStruct = { 0 };
    RCC_ClkInitTypeDef RCC_ClkInitStruct = { 0 };
    if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1)
        != HAL_OK) {
        Error_Handler();
    }
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI
        | RCC_OSCILLATORTYPE_MSI;
    RCC_OscInitStruct.HSISState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.MSISState = RCC_MSI_ON;
    RCC_OscInitStruct.MSICalibrationValue = RCC_MSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_6;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = 2;
    RCC_OscInitStruct.PLL.PLLN = 20;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV7;
    RCC_OscInitStruct.PLL.PLLQ = RCC_PLLQ_DIV2;
    RCC_OscInitStruct.PLL.PLLR = RCC_PLLR_DIV2;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) {
        Error_Handler();
    }
}

```

```

    }
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK
        | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4) != HAL_OK) {
        Error_Handler();
    }
}

```

CAN1.h

```

/* USER CODE BEGIN Header */
/*****
 * EE 329 OBD2 Reader and Display
 *****/
 * @file      : CAN1.h
 * @brief     : Contains function prototypes for CAN1.c
 * project    : EE 329 S'25 OBD2 Project
 * authors    : Mark Negus (MN) - mnegus@calpoly.edu
 *            : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * version    : 1.0
 * date       : 060125
 * compiler   : STM32CubeIDE v.1.18.0
 * target     : NUCLEO-L496ZG
 * clocks     : 80 MHz
 * @attention : © 2025 STMicroelectronics. All rights reserved.
 *****/
 * 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-234567 */
/* USER CODE END Header */
#ifndef FDCAN_H
#define FDCAN_H
#include <stdint.h>
void FDCAN_Init(void);
void FDCAN_Send(uint16_t std_id, const uint8_t *data, uint8_t len);
uint8_t FDCAN_Receive(uint16_t *std_id, uint8_t *data);
#endif

```

CAN1.c

```

/* USER CODE BEGIN Header */
/*****
 * EE 329 OBD2 Reader and Display
 *****/
 * @file      : CAN1.c
 * @brief     : Contains functions to initialize CAN, request CAN data, and
 *             handle the received CAN data
 * project    : EE 329 S'25 OBD2 Project
 * authors    : Mark Negus (MN) - mnegus@calpoly.edu
 *             Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * version    : 1.0
 * date       : 06/01/25
 * compiler   : STM32CubeIDE v.1.18.0
 * target     : NUCLEO-L496ZG
 * clocks     : 80 MHz
 * @attention : © 2025 STMicroelectronics. All rights reserved.
 *****/
 * 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-2345678 */
/* USER CODE END Header */
#include "stm32l4xx.h"
#include <string.h>
#include "CAN1.h"

/*-----
 * function : CAN1_Init_500k()
 * author   : Mark Negus (MN) - mnegus@calpoly.edu
 *           Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * input(s) : none
 * output(s) : none
 * action   : Initialize CAN1 with baud rate of 500kHz
 * version  : 1.0
 * date     : 06/01/25
 *-----*/

extern void CAN1_Init_500k(void) {
    RCC->APB1ENR1 |= RCC_APB1ENR1_CAN1EN; // 80MHz clock
    __DSB(); // Data Sync
    CAN1->MCR |= CAN_MCR_INRQ; // request initialization from MCR

    while (!(CAN1->MSR & CAN_MSR_INAK))
        ;
    CAN1->MCR &= ~CAN_MCR_SLEEP;
    CAN1->MCR |= CAN_MCR_ABOM; // bus management
    CAN1->MCR |= CAN_MCR_AWUM; // wake up mode
    CAN1->MCR &= ~CAN_MCR_NART; // retransmission enabled
    // BTR = bit timing register, must be set to 500kbs
    // (BRP=10, TS1=11, TS2=4, SJW=1. Total TQ = 16. 80MHz / 10 / 16 = 500kHz)
    CAN1->BTR = (0 << CAN_BTR_SJW_Pos) | (3 << CAN_BTR_TS2_Pos)
        | (10 << CAN_BTR_TS1_Pos) | (9 << CAN_BTR_BRP_Pos);
    CAN1->BTR &= ~(CAN_BTR_LBKM | CAN_BTR_SILM);
    // Filter Mask Register initialization
    CAN1->FMR |= CAN_FMR_FINIT;
    CAN1->FM1R &= ~(1 << 0);
    CAN1->FS1R |= (1 << 0); // 32 bit filter
    CAN1->FFA1R &= ~(1U << 0);
    //filter registers

```

```

CAN1->sFilterRegister[0].FR1 = 0x00000000; // accept all
CAN1->FA1R |= (1U << 0); // Activate filter 0
CAN1->FMR &= ~CAN_FMR_FINIT; // exit FMR initialization
CAN1->MCR &= ~CAN_MCR_INRQ; // exit initialization mode
}

/*-----*/
* function : CAN1_Send()
* author   : Mark Negus (MN) - mnegus@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : id (uint32_t), *data (uint8_t), length (uint8_t)
* output(s): int
* action   : Send CAN data request to car/simulated car ECU
* version  : 1.0
* date     : 06/01/25
*-----*/
extern int CAN1_Send(uint32_t id, const uint8_t *data, uint8_t length) {
    if (length > 8) { //can only do classic can
        return 0;
    }
    uint8_t mailbox = 0;
    //find empty tx mailbox
    if (CAN1->TSR & CAN_TSR_TME0) {
        mailbox = 0;
    } else if (CAN1->TSR & CAN_TSR_TME1) {
        mailbox = 1;
    } else if (CAN1->TSR & CAN_TSR_TME2) {
        mailbox = 2;
    } else {
        return 0; //no mailbox
    }
    CAN_TxMailBox_TypeDef *tx_mailbox = &CAN1->sTxMailBox[mailbox];
    // Set 11 bit ID
    // TIR = transmit identifier reg
    tx_mailbox->TIR = ((id & 0x7FF) << CAN_TI0R_STID_Pos);
    // Set data length (DLC = data length code, a code
    // in the frame indicating how many data bits)
    // TDTR = transmit data length/time reg
    tx_mailbox->TDTR = (length & CAN_TDT0R_DLC_Msk);
    // copy data
    // 8 bytes of data is split into two 4 byte regs, high and low
    // these are temp storage
    uint32_t data_low = 0, data_high = 0;
    //memcpy is built in via library - copies the high byte if necessary
    // memcpy format: (destination address, source, size in bytes)
    if (length > 0)
        memcpy(&data_low, data, (length < 4) ? length : 4);
    if (length > 4)
        memcpy(&data_high, data + 4, length - 4);
    tx_mailbox->TDLR = data_low; //TDLR = transmit data low (low byte)
    tx_mailbox->TDHR = data_high; // transmit data high
    // request transmission
    tx_mailbox->TIR |= CAN_TI0R_TXRQ;
    return 1; //Success
}

```

```

/*-----
 * function : CAN1_Receive()
 * author   : Mark Negus (MN) - mnegus@calpoly.edu
 *           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * input(s) : *id (uint32_t), *data (uint8_t), *length (uint8_t)
 * output(s): int
 * action   : Handle received CAN data from car/simulated car ECU
 * version  : 1.0
 * date     : 06/01/25
 *-----*/
extern int CAN1_Receive(uint32_t *id, uint8_t *data, uint8_t *length) {
    // check if FIFO0 for messages
    if ((CAN1->RF0R & CAN_RF0R_FMP0_Msk) == 0) {
        return 0;
    }
    CAN_FIFOMailBox_TypeDef *rx_mailbox = &CAN1->sFIFOMailBox[0];
    // get ID
    // use 0x7FF to get only the lowest 2047 (11 ID bits)
    //RIR = receive identifier reg
    *id = (rx_mailbox->RIR >> CAN_RI0R_STID_Pos) & 0x7FF;
    // get data length
    *length = (uint8_t) (rx_mailbox->RDTR & CAN_RDT0R_DLC_Msk);
    if (*length > 8)
        *length = 8;
    // get data
    uint32_t data_low = rx_mailbox->RDLR; // lower data byte
    uint32_t data_high = rx_mailbox->RDHR; // higher data byte
    memcpy(data, &data_low, (*length < 4) ? *length : 4);
    if (*length > 4) {
        memcpy(data + 4, &data_high, *length - 4);
    }
    // release FIFO - now the reg is able to update again
    CAN1->RF0R |= CAN_RF0R_RFOM0;
    return 1;
}

```

LPUART.h

```

/* USER CODE BEGIN Header */
/*****
 * EE 329 OBD2 Reader and Display
 *****/
 * @file           : LPUART.h
 * @brief          : Contains function prototypes for LPUART.c
 * project         : EE 329 S'25 OBD2 Project
 * authors         : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * version         : 1.0
 * date            : 050725
 * compiler        : STM32CubeIDE v.1.18.0

```

```

* target          : NUCLEO-L496ZG
* clocks          : 80 MHz
* @attention      : © 2025 STMicroelectronics. All rights reserved.
*****
* 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-2345678 */
/* USER CODE END Header */
#include <stdint.h>
#include <stdio.h>
#include "stm32l4xx.h"
#ifndef SRC_LPUART_H_
#define SRC_LPUART_H_
void LPUART_init(void);
void LPUART_Print(const char* message);
void LPUART_ESC_Print(const char* command);
void LPUART1_IRQHandler(void);
#endif /* SRC_LPUART_H_ */

```

LPUART.c

```

/* USER CODE BEGIN Header */
/*****
* EE 329 OBD2 Reader and Display
*****
* @file          : LPUART.c
* @brief         : Contains functions to initialize LPUART, LPUART print,
*                 and handle LPUART interrupt request
* project        : EE 329 S'25 OBD2 Project
* authors        : Bryan Lew (BL) - bglew@calpoly.edu
* version        : 1.0
* date          : 05/20/25
* compiler       : STM32CubeIDE v.1.18.0
* target        : NUCLEO-L496ZG
* clocks        : 80 MHz
* @attention     : © 2025 STMicroelectronics. All rights reserved.
*****
* 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-2345678 */
/* USER CODE END Header */
#include "LPUART.h"
/*-----
* function : LPUART_init()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
* input(s) : none
* output(s): none
* action   : Initialize LPUART registers & adjusted the baud rate to account
*           for 80MHz clock
* version  : 1.0
* date     : 05/20/25
*-----*/
void LPUART_init(void) {

```

```

PWR->CR2 |= (PWR_CR2_IOSV); //power avail on PG[15:2] (LPUART1)
RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOGEN); //enable GPIOG clock
RCC->APB1ENR2 |= RCC_APB1ENR2_LPUART1EN; //enable LPUART clock bridge
// Set PG7 to AF8 (USART) function
GPIOG->AFR[0] &= ~(0xF << GPIO_AFRL_AFSEL7_Pos); // Clear PG7 AF
GPIOG->AFR[0] |= (0x8 << GPIO_AFRL_AFSEL7_Pos); // Sets PG7 AF
// Set PG8 to AF8 (USART) function
GPIOG->AFR[1] &= ~(0xF << GPIO_AFRH_AFSEL8_Pos); // Clear PG8 AF
GPIOG->AFR[1] |= (0x8 << GPIO_AFRH_AFSEL8_Pos); // Sets PG8 AF
GPIOG->OTYPER &= ~(GPIO_OTYPER_OT7 | GPIO_OTYPER_OT8); // Push-pull
GPIOG->PUPDR &= ~(GPIO_PUPDR_PUPD7 | GPIO_PUPDR_PUPD8); // No pull-up/down
GPIOG->OSPEEDR |= (GPIO_OSPEEDER_OSPEEDR7 |
GPIO_OSPEEDER_OSPEEDR8); // High speed
GPIOG->MODER &= ~(GPIO_MODER_MODE7 | GPIO_MODER_MODE8); // Clear mode
GPIOG->MODER |= (GPIO_MODER_MODE7_1 |
GPIO_MODER_MODE8_1); // Set to AF mode (10)
LPUART1->CR1 &= ~(USART_CR1_M1 | USART_CR1_M0); // 8-bit data
LPUART1->CR1 |= USART_CR1_UE; // enable LPUART1
LPUART1->CR1 |= (USART_CR1_TE | USART_CR1_RE); // enable xmit & recv
LPUART1->CR1 |= USART_CR1_RXNEIE; // enable LPUART1 recv interrupt
LPUART1->ISR &= ~(USART_ISR_RXNE); // clear Recv-Not-Empty flag
LPUART1->BRR = (256 * 4000000) / 115200; //baud rate=256*CLK_rate/UART freq
NVIC->ISER[2] = (1 << (LPUART1_IRQn & 0x1F)); // enable LPUART1 ISR
__enable_irq(); // enable global interrupts
LPUART_ESC_Print("[2J\033[H"); //clear the screen
}

/*-----
* function : LPUART_Print()
* author : Bryan Lew (BL) - bglew@calpoly.edu
* input(s) : *message (const char)
* output(s): none
* action : Print message to LPUART
* version : 1.0
* date : 05/20/25
*-----*/
void LPUART_Print(const char *message) {
    uint16_t iStrIdx = 0;
    while (message[iStrIdx] != 0) {
        while (!(LPUART1->ISR & USART_ISR_TXE))
            // wait for empty xmit buffer
        ;
        LPUART1->TDR = message[iStrIdx]; // send this character
        iStrIdx++; // advance index to next char
    }
}

/*-----
* function : LPUART1_IRQHandler()
* author : Bryan Lew (BL) - bglew@calpoly.edu
* input(s) : none
* output(s): none
* action : Initialize interrupt request handler for LPUART
* version : 1.0
* date : 05/20/25

```

```

*-----*/
void LPUART1_IRQHandler(void) {
    /* clear overrun if it occurred */
    if (LPUART1->ISR & USART_ISR_ORE)
        LPUART1->ICR = USART_ICR_ORECF;
    if (LPUART1->ISR & USART_ISR_RXNE)           // byte received?
    {
        uint8_t ch = (uint8_t) LPUART1->RDR;      // read clears RXNE
        while (!(LPUART1->ISR & USART_ISR_TXE))
            ;
        LPUART1->TDR = ch;
    }
}

```

lcd.h

```

/* USER CODE BEGIN Header */
/*****
 * EE 329 OBD2 Reader and Display
 *****/
* @file      : lcd.h
* @brief     : Contains function prototypes and definitions for LCD.c
* project    : EE 329 S'25 OBD2 Project
* authors    : Bryan Lew (BL) - bglew@calpoly.edu
*            : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* version    : 1.0
* date       : 05/12/25
* compiler   : STM32CubeIDE v.1.18.0
* target     : NUCLEO-L496ZG
* clocks     : 80 MHz
* @attention : © 2025 STMicroelectronics. All rights reserved.
*****/
* 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-234567 */
/* USER CODE END Header */
#ifndef INC_LCD_H_
#define INC_LCD_H_
#include "main.h"
#include <stdint.h>
// LCD functions to be available for future projects
void LCD_config(void);
////////////////////////////////////////////////////
void LCD_init(void);           // initialize LCD
void LCD_pulse_ENA(void);
void LCD_CursorBlinkOn(void);
void LCD_4b_command(uint8_t command);
void LCD_command(uint8_t command); // send LCD a single 8-bit command
void LCD_write_char(uint8_t letter); // write a character to the LCD

```

```

void LCD_start_msg(); //write start message to LCD
void LCD_react_time(int ms); //display reaction time to LCD
////////////////////////////////////
// printing a string to the LCD that's constant and won't be changed
void LCD_write_string(const char *str);
//position to start writing on LCD
void LCD_setcursor(uint8_t row, uint8_t col);
//defining port E for LCD
#define LCD_PORT GPIOE
//defining pin for RS RW E/N
#define LCD_PIN_RS GPIO_PIN_2
#define LCD_PIN_RW GPIO_PIN_3
#define LCD_PIN_EN GPIO_PIN_4
// defining things from professors provided code
#define LCD_DATA_BITS (0x0F << 5 ) //PE5 to PE8
// LCD commands from ST7066U data sheet
#define LCD_clear_display (0x01) // 0000x0001
#define LCD_set_cursor (0x10)
#define LCD_return_home (0x02) // 0000x0010
#define LCD_entry_mode_set (0x06) // 0000x0111
#define LCD_display_on (0x0F) // 0000x1111
#define LCD_function_set (0x28) // 0010x1000
//when the COUNTDOWN is complete turn ON an LED
//and leave it ON until the '#' is pressed.
void LCD_countdown_led_on(void);
#endif /* INC_LCD_H_ */

```

lcd.c

```

/* USER CODE BEGIN Header */
/*****
 * EE 329 OBD2 Reader and Display
 *****/
 * @file      : lcd.c
 * @brief     : Contains functions to configure, initialize and write
 *             to the LCD
 * project    : EE 329 S'25 OBD2 Project
 * authors    : Bryan Lew (BL) - bglew@calpoly.edu
 *             Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * version    : 1.0
 * date       : 05/12/25
 * compiler   : STM32CubeIDE v.1.18.0
 * target     : NUCLEO-L496ZG
 * clocks     : 80 MHz
 * @attention : © 2025 STMicroelectronics. All rights reserved.
 *****/
 * 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-234567 */
/* USER CODE END Header */

```

```

#include "lcd.h"
#include "delay.h"
#include "main.h"
#include "string.h"
#include "stdio.h"

/*-----
 * function : LCD_config()
 * author   : Bryan Lew (BL) - bglew@calpoly.edu
 *           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * input(s) : none
 * output(s) : none
 * action    : Configure GPIO for LCD
 * version   : 1.0
 * date      : 05/12/25
 *-----*/
void LCD_config(void) {
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOEEN; // enabling clock for port E
    //clear mode
    LCD_PORT->MODER &= ~(GPIO_MODER_MODE2 |
    GPIO_MODER_MODE3 |
    GPIO_MODER_MODE4 |
    GPIO_MODER_MODE5 |
    GPIO_MODER_MODE6 |
    GPIO_MODER_MODE7 |
    GPIO_MODER_MODE8);
    //set mode
    LCD_PORT->MODER |= (GPIO_MODER_MODE2_0 |
    GPIO_MODER_MODE3_0 |
    GPIO_MODER_MODE4_0 |
    GPIO_MODER_MODE5_0 |
    GPIO_MODER_MODE6_0 |
    GPIO_MODER_MODE7_0 |
    GPIO_MODER_MODE8_0);
    //output drive
    LCD_PORT->OTYPER &= ~(GPIO_OTYPER_OT2 |
    GPIO_OTYPER_OT3 |
    GPIO_OTYPER_OT4 |
    GPIO_OTYPER_OT5 |
    GPIO_OTYPER_OT6 |
    GPIO_OTYPER_OT7 |
    GPIO_OTYPER_OT8);
    //pull-up/down resistors
    LCD_PORT->PUPDR &= ~(GPIO_PUPDR_PUPD2 |
    GPIO_PUPDR_PUPD3 |
    GPIO_PUPDR_PUPD4 |
    GPIO_PUPDR_PUPD5 |
    GPIO_PUPDR_PUPD6 |
    GPIO_PUPDR_PUPD7 |
    GPIO_PUPDR_PUPD8);
    //speed
    LCD_PORT->OSPEEDR &= ~(3 << GPIO_OSPEEDR_OSPEED2_Pos
    | 3 << GPIO_OSPEEDR_OSPEED3_Pos | 3 << GPIO_OSPEEDR_OSPEED4_Pos
    | 3 << GPIO_OSPEEDR_OSPEED5_Pos | 3 << GPIO_OSPEEDR_OSPEED6_Pos
    | 3 << GPIO_OSPEEDR_OSPEED7_Pos | 3 << GPIO_OSPEEDR_OSPEED8_Pos);

```

```

//reset pins
LCD_PORT->BRR = (GPIO_MODER_MODE2 |
GPIO_MODER_MODE3 |
GPIO_MODER_MODE4 |
GPIO_MODER_MODE5 |
GPIO_MODER_MODE6 |
GPIO_MODER_MODE7 |
GPIO_MODER_MODE8);
}

/*-----*/
* function : LCD_init()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : none
* output(s): none
* action   : Initialize LCD
* version  : 1.0
* date     : 05/12/25
*-----*/
void LCD_init(void) { //RCC & GPIO config removed - leverage A1, A2 code
    delay_us(40000); //power-up wait 40 MS
    for (int idx = 0; idx < 3; idx++) { //wake up 1,2,3: DATA = 0011 XXXX
        LCD_4b_command(0x30); //HI 4b of 8b cmd, low nibble = X
        delay_us(200);
    }
    LCD_4b_command(0x20); // SWITCHING TO 4 BIT NIBBLE MODE
    // fcn set #4: 4b cmd set 4b mode - next 0x28:2-line
    delay_us(40); // remainder of LCD init removed - see LCD datasheets
    LCD_command(LCD_function_set);
    delay_us(40);
    LCD_command(LCD_set_cursor);
    delay_us(40);
    LCD_command(LCD_display_on); //turn on display, cursor, enable blinking
    delay_us(40);
    LCD_command(LCD_entry_mode_set);
    delay_us(40);
    LCD_command(LCD_clear_display); // clears display
    delay_us(2000); // 2 MS
    LCD_command(LCD_return_home);
    delay_us(40);
    LCD_command(0x80); // 1000x0000 TELLING TO START AT LINE 1 OF DDRAM ADDRESS
    delay_us(40);
}

/*-----*/
* function : LCD_pulse_ENA()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : none
* output(s): none
* action   : Enable pulse for LCD
* version  : 1.0
* date     : 05/12/25

```

```

/*-----*/
void LCD_pulse_ENA(void) {
    // ENAbLe line sends command on falling edge
    // set to restore default then clear to trigger
    LCD_PORT->ODR |= ( LCD_PIN_EN);    // ENABLE = HI
    delay_us(20);                      // TDDR > 320 ns
    LCD_PORT->ODR &= ~( LCD_PIN_EN);    // ENABLE = LOW
    delay_us(20);                      // low values flakey, see A3:p.1
}

/*-----*/
* function : LCD_init()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : none
* output(s): none
* action   : Configure LCD to 4 bit input
* version  : 1.0
* date     : 05/12/25
*-----*/
void LCD_4b_command(uint8_t command) {
    // LCD command using high nibble only - used for 'wake-up' 0x30 commands
    LCD_PORT->ODR &= ~( LCD_DATA_BITS);    // clear DATA bits
    LCD_PORT->ODR |= (command >> 4);       // DATA = command
    delay_us(20);
    LCD_pulse_ENA();
}

/*-----*/
* function : LCD_command()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : command (uint8_t)
* output(s): none
* action   : Take in LCD command and configure bits
* version  : 1.0
* date     : 05/12/25
*-----*/
void LCD_command(uint8_t command) {
    delay_us(2000);    // Delay for LCD clear command (important!)
    // send command to LCD in 4-bit instruction mode
    // HIGH nibble then LOW nibble, timing sensitive
    LCD_PORT->ODR &= ~( LCD_DATA_BITS);    // isolate cmd bits
    LCD_PORT->ODR |= ((command >> 4) & 0x0F) << 5; // HIGH shifted low
    delay_us(20);
    LCD_pulse_ENA();    // latch HIGH NIBBLE
    delay_us(400);
    LCD_PORT->ODR &= ~( LCD_DATA_BITS);    // isolate cmd bits
    LCD_PORT->ODR |= (command & 0x0F) << 5; // LOW nibble
    delay_us(20);
    LCD_pulse_ENA();    // latch LOW NIBBLE
    delay_us(400);
}

/*-----*/

```

```

* function : LCD_write_char()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : letter (uint8_t)
* output(s): none
* action    : Initialize LCD
* version   : 1.0
* date      : 05/12/25
*-----*/
void LCD_write_char(uint8_t letter) {
    LCD_PORT->ODR |= (LCD_PIN_RS); // RS = HI for data to address SET EQUAL 1
    LCD_PORT->ODR &= ~(LCD_PIN_RW); // RW = 0 for data
    delay_us(20);
    LCD_command(letter); // character to print
    LCD_PORT->ODR &= ~(LCD_PIN_RS); // RS = LO
}

/*-----*/
* function : LCD_setcursor()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : row (uint8_t), col (uint8_t)
* output(s): none
* action    : Set cursor for LCD
* version   : 1.0
* date      : 05/12/25
*-----*/
void LCD_setcursor(uint8_t row, uint8_t col) {
    uint8_t address;
    //switch statement to select top or bottom row
    switch (row) {
        //top row
        case (0):
            address = 0x80 + col;
            break;
        //bot row
        case (1):
            address = 0xC0 + col;
            break;
        default:
            return;
    }
    LCD_command(address);
    delay_us(40); //40us
}

/*-----*/
* function : LCD_write_string()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : *str (const char)
* output(s): none
* action    : Write string to LCD function
* version   : 1.0
* date      : 05/12/25

```

```

/*-----*/
void LCD_write_string(const char *str) {
    //write each individual character of string in for loop
    for (int i = 0; i < strlen(str); i++) {
        LCD_write_char(str[i]);
        delay_us(50);
    }
}

/*-----
* function : LCD_start_msg()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : none
* output(s): none
* action   : Display start msg on initial start-up before car connection
* version  : 1.0
* date     : 05/12/25
*-----*/
void LCD_start_msg(void) {
    LCD_config(); //enables GPIO pins for LCD
    LCD_init();   //wake up + 4 bit set
    LCD_command(0x01);
    delay_us(2000);
    LCD_setcursor(0, 0);
    LCD_write_string("READ ERROR");
    LCD_setcursor(1, 0);
    delay_us(2000);
    LCD_write_string("TURN ON CAR");
}

/*-----
* function : LCD_Update()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : ms (int)
* output(s): none
* action   : Update LCD with MPH and RPM
* version  : 1.0
* date     : 05/12/25
*-----*/
void LCD_Update(int ms) {
    char str[18];
    //generate string to print to LCD
    sprintf(str, "MPH %d RPM %d", getMPH(), getRPM());
    LCD_setcursor(1, 0); //set cursor in correct location
    LCD_write_string(str); //print to LCD
    delay_us(400);
}

```

led.h

```

/* USER CODE BEGIN Header */
/*****
 * EE 329 OBD2 Reader and Display
 *****/

 * @file      : led.h
 * @brief     : Contains function prototypes and definitions for LED.c
 * project    : EE 329 S'25 OBD2 Project
 * authors    : Bryan Lew (BL) - bglew@calpoly.edu
 *            : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * version    : 1.0
 * date       : 05/20/25
 * compiler   : STM32CubeIDE v.1.18.0
 * target     : NUCLEO-L496ZG
 * clocks     : 80 MHz
 * @attention : © 2025 STMicroelectronics. All rights reserved.
 *****/
 * 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-234567 */
/* USER CODE END Header */

#ifndef LED_H
#define LED_H
#include <stdint.h> // For uint8_t, uint16_t
#include <stdbool.h> // For bool
#define SHIFT_RPM_THRESHOLD 6000 // Ex RPM for the individual shift light
#define MAX_RPM 7000 // Ex Max RPM for full bar display/flashing
#define SHIFT_LED_PIN 12 // PB12 for the dedicated shift light
#define LED_COUNT 6 // # of LEDs for the rpm display (PB0-PB5)
// Function prototypes
extern void ShiftLight_Init(void);
extern void DisplayRPM(uint16_t rpm);
#endif // LED_H

```

led.c

```

/* USER CODE BEGIN Header */
/*****
 * EE 329 OBD2 Reader and Display
 *****/

 * @file      : led.c
 * @brief     : Contains functions to initialize rpm lights and map
 *            : the engine rpm to the individual LEDs
 * project    : EE 329 S'25 OBD2 Project
 * authors    : Bryan Lew (BL) - bglew@calpoly.edu
 *            : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
 * version    : 1.0
 * date       : 05/20/25
 * compiler   : STM32CubeIDE v.1.18.0
 * target     : NUCLEO-L496ZG
 * clocks     : 80 MHz

```

```

* @attention      : © 2025 STMicroelectronics. All rights reserved.
*****
* 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-2345678 */
/* USER CODE END Header */
#include "led.h"
#include "stm32l4xx.h" // For GPIO register access
#include "delay.h"      // For delay_us (will be used directly and blockingly)
#include <stdint.h>      // For uint8_t, uint16_t
#include <stdbool.h>     // For bool

/*-----*/
* function : ShiftLight_Init()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s) : none
* output(s): none
* action    : Initialize GPIO for LEDs in rpm light and shift light
* version   : 1.0
* date      : 05/18/25
*-----*/
extern void ShiftLight_Init(void) {
    // Enable GPIOB clock
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;
    for (uint8_t pin = 0; pin <= 5; pin++) //Initialize PB0 through PB5 for bar
    {
        GPIOB->MODER &= ~(3U << (pin * 2)); // Clear mode bits (0b11)
        GPIOB->MODER |= (1U << (pin * 2)); // Set as output (01)
        GPIOB->OTYPER &= ~(1U << pin); // Push-pull (0)
        GPIOB->PUPDR &= ~(3U << (pin * 2)); // No pull-up/pull-down (00)
        GPIOB->OSPEEDR |= (3U << (pin * 2)); // High speed (0b11)
    }
    if (SHIFT_LED_PIN > 5) {
        GPIOB->MODER &= ~(3U << (SHIFT_LED_PIN * 2));
        GPIOB->MODER |= (1U << (SHIFT_LED_PIN * 2));
        GPIOB->OTYPER &= ~(1U << SHIFT_LED_PIN);
        GPIOB->PUPDR &= ~(3U << (SHIFT_LED_PIN * 2));
        GPIOB->OSPEEDR |= (3U << (SHIFT_LED_PIN * 2));
    }
}

/*-----*/
* function : DisplayRPM()
* author    : Bryan Lew (BL) - bglew@calpoly.edu
*           : Hunter Pinkstaff (HP) - hpinksta@calpoly.edu
* input(s)  : rpm (uint16_t)
* output(s) : none
* action    : Take in an rpm value, map it, and display on LEDs
* version   : 1.0
* date      : 05/18/25
*-----*/
extern void DisplayRPM(uint16_t rpm) {
    const uint16_t BAR_LED_MASK = 0x3F;
    if (rpm >= SHIFT_RPM_THRESHOLD)
        GPIOB->ODR |= (1U << SHIFT_LED_PIN); // Turn on shiftlight
    else

```

```

    GPIOB->ODR &= ~(1U << SHIFT_LED_PIN); // Turn off shiftlight
    if (rpm >= MAX_RPM) // Redline condition
    {
        GPIOB->ODR |= BAR_LED_MASK;
        delay_us(2000);
        GPIOB->ODR &= ~BAR_LED_MASK;
        delay_us(2000);
    } else {
        GPIOB->ODR &= ~BAR_LED_MASK;
        uint8_t leds_on = (rpm * LED_COUNT) / MAX_RPM;
        if (leds_on > LED_COUNT)
            leds_on = LED_COUNT;
        if (leds_on > 0)
            GPIOB->ODR |= ((1U << leds_on) - 1);
    }
}

```

delay.h

```

/* USER CODE BEGIN Header */
/*****
 * EE 329 OBD2 Reader and Display
 *****/
 * @file      : delay.h
 * @brief     : Contains function prototype for delay.c
 * project    : EE 329 S'25 OBD2 Project
 * authors    : Bryan Lew (BL) - bglew@calpoly.edu
 * version    : 1.0
 * date       : 05/18/25
 * compiler   : STM32CubeIDE v.1.18.0
 * target     : NUCLEO-L496ZG
 * clocks     : 80 MHz
 * @attention : © 2025 STMicroelectronics. All rights reserved.
 *****/
 * 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-234567 */
/* USER CODE END Header */
#ifndef SRC_DELAY_H_
#define SRC_DELAY_H_

#include <stdint.h>
#include "stm32l4xx.h"
extern void delay_us(const uint32_t time_us);

#endif /* SRC_DELAY_H_ */

```

delay.c

```
-----
/* USER CODE BEGIN Header */
/*****
 * EE 329 OBD2 Reader and Display
 *****/
 * @file      : delay.c
 * @brief     : Contains functions to create microsecond delay w/ SysTick
 * project    : EE 329 S'25 OBD2 Project
 * authors    : Bryan Lew (BL) - bglew@calpoly.edu
 * version    : 1.0
 * date       : 05/18/25
 * compiler   : STM32CubeIDE v.1.18.0
 * target     : NUCLEO-L496ZG
 * clocks     : 80 MHz
 * @attention : © 2025 STMicroelectronics. All rights reserved.
 *****/
 * 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-234567 */
/* USER CODE END Header */
#include "delay.h"
#include "stm32l4xx.h"
/*-----
 * function : SysTick_Init()
 * author   : Bryan Lew (BL) - bglew@calpoly.edu
 * input(s) : none
 * output(s): none
 * action   : Initialize SysTick to create timer based off core clock
 * version  : 1.0
 * date     : 05/18/25
 *-----*/
void SysTick_Init(void) {
    SysTick->CTRL |= (SysTick_CTRL_ENABLE_Msk |
                     SysTick_CTRL_CLKSOURCE_Msk); // enable SysTick
    SysTick->CTRL &= ~SysTick_CTRL_TICKINT_Msk;    // disable SysTick IRQ
}

/*-----
 * function : delay_us()
 * author   : Bryan Lew (BL) - bglew@calpoly.edu
 * input(s) : time_us (uint32_t)
 * output(s): none
 * action   : Generates block delay in microseconds using SysTick timer
 * version  : 1.0
 * date     : 05/18/25
 *-----*/
void delay_us(const uint32_t time_us) {
    if (time_us == 0)
        return;
    // Calculate timer reload value from system clock
    SysTick->LOAD = (uint32_t) ((time_us * (SystemCoreClock / 1000000)) - 1);
    SysTick->VAL = 0; // clear current count
    SysTick->CTRL &= ~SysTick_CTRL_COUNTFLAG_Msk; // clear overflow flag
}
```

```
// Wait for COUNTFLAG to set when timer expires
while (!(SysTick->CTRL & SysTick_CTRL_COUNTFLAG_Msk))
    ;
}
```

TRANSMITTER STM32

main.h

```
/**
*****
* @file      : main.h
* @project   : EE 329 Simulated CAR ECU & CAN Transmitter
* @author    : Bryan Lew
* @date      : 060525
* @brief     : Contains definitions & prototypes for OBD-II, CAN, & test values
*****
*/
/* Define to prevent recursive inclusion -----*/
#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C" {
#endif

#include "stm32l4xx_hal.h"

void Error_Handler(void);

//OBD-II constants
#define PID_ENGINE_RPM      0x0C
#define PID_COOLANT_TEMP    0x05
#define PID_VEHICLE_SPEED  0x0D

#define OBD_REQUEST_ID      0x7DF //functional request
#define OBD_RESPONSE_ID    0x7E8 //ECU-1 response

//simulated sensor values
#define RPM_START    800 //start at 800 rpm
#define TEMP_START   20  //start at 20 deg C
#define SPEED_START  0    //start at 0 km/h

#define RPM_MAX      7500 //max rpm
#define TEMP_MAX      110 //max temp
#define SPEED_MAX     180 //max speed

#define RPM_STEP      100 //rpm step
#define TEMP_STEP      2  //temp step
#define SPEED_STEP     5  //speed step

//prototypes
void SystemClock_Config(void);
void CAN1_Init(void);
```

```

void CAN1_SendFrame(uint32_t id, uint8_t dlc, const uint8_t *data);
void Process_OBD_Request(uint32_t id, uint8_t dlc, const uint8_t *data);
void Update_Dynamic_Values(void);

```

```

#ifdef __cplusplus
}
#endif

```

```

#endif /* __MAIN_H */

```

main.c

```

/* USER CODE BEGIN Header */
/*****
 * EE 329 Simulated CAR ECU & CAN Transmitter
 *****/
* @file      : main.c
* @brief     : Emulates a single OBD-II ECU that answers generic
*              mode-01 PID requests for rpm, coolant temperature,
*              and vehicle speed. The three values ramp upward and
*              roll over, giving a scan-tool "live" data to display.
* project    : EE 329 S'25 OBD2 Project
* authors    : Bryan Lew (BL) - bglew@calpoly.edu
* version    : 1.3
* date       : 06/05/25
* compiler   : STM32CubeIDE v.1.18.0
* target     : NUCLEO-L4A6ZG
* clocks     : 80 MHz
* @attention : © 2025 STMicroelectronics. All rights reserved.
*****/
* PLAN
* 1. Write #defines, globals, prototypes
* 2. Configure clocks -> 80 MHz SYSClk
* 3. Initialize OBD-II PIDs and logic
* 4. Main loop: every 200 ms bump simulated sensor values
* 5. On CAN RX: parse 0x7DF functional query and respond on 0x7E8
*****/
* REVISION HISTORY
* 1.0 Initial implementation with static rpm, coolant temp, vehicle speed
* 1.1 Fixed PIDs and CAN pins, rewrote OBD-II and CAN functions
* 1.2 Made rpm, coolant temp, and vehicle speed have dynamic values
* 1.3 Commented & reformatted code
*****/
* 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-234567 */
/* USER CODE END Header */

//includes
#include "stm32l4a6xx.h"

```

```

#include <string.h>
#include "delay.h"
#include "main.h"

uint16_t current_rpm = RPM_START;
int8_t current_coolant_temp = TEMP_START;
uint8_t current_speed_kph = SPEED_START;

/*-----*/
* function : main()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
* input(s) : none
* output(s): int
* action   : Handles the update and delays for CAN
* version  : 1.0
* date     : 06/05/25
*-----*/
int main(void) {
    SystemClock_Config();
    SysTick_Init(); //initialize SysTick
    CAN1_Init();    //initialize CAN1

    const uint32_t UPDATE_INTERVAL_MS = 200; //update interval
    uint32_t tick = 0;

    while (1) {
        if (tick >= UPDATE_INTERVAL_MS) {
            Update_Dynamic_Values();
            tick = 0;
        } else {
            tick++;
        }
        delay_us(1000); //1 ms delay
    }
}

/*-----*/
* function : Update_Dynamic_Values()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
* action   : Increment each simulated sensor, wrap when max is reached.
* version  : 1
* date     : 06/05/25
*-----*/
void Update_Dynamic_Values(void) {
    //rpm
    current_rpm += RPM_STEP;
    if (current_rpm > RPM_MAX)
        current_rpm = RPM_START;

    //coolant temperature
    current_coolant_temp += TEMP_STEP;
    if (current_coolant_temp > TEMP_MAX)
        current_coolant_temp = TEMP_START;

    //vehicle speed

```

```

    current_speed_kph += SPEED_STEP;
    if (current_speed_kph > SPEED_MAX)
        current_speed_kph = SPEED_START;
}

/*-----
 * function : CAN1_Init()
 * author   : Bryan Lew (BL) - bglew@calpoly.edu
 * action    : Configure PB8/PB9 for AF9, set bit-timing, enable filter 0
 *            (accept-all), enable FIFO0 interrupt.
 * version   : 1
 * date      : 06/05/25
 *-----*/
void CAN1_Init(void) {
    volatile uint32_t t;

    //reset peripheral
    RCC->APB1RSTR1 |= RCC_APB1RSTR1_CAN1RST;
    for (t = 0; t < 200; t++)
        ;
    RCC->APB1RSTR1 &= ~RCC_APB1RSTR1_CAN1RST;

    //configure high-speed push-pull
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;
    GPIOB->AFR[1] &= ~(GPIO_AFRH_AFSEL8_Msk | GPIO_AFRH_AFSEL9_Msk);
    GPIOB->AFR[1] |= (9U << GPIO_AFRH_AFSEL8_Pos) | (9U << GPIO_AFRH_AFSEL9_Pos);
    GPIOB->MODER &= ~(GPIO_MODER_MODE8_Msk | GPIO_MODER_MODE9_Msk);
    GPIOB->MODER |= (GPIO_MODER_MODE8_1 | GPIO_MODER_MODE9_1);
    GPIOB->OTYPER &= ~(GPIO_OTYPER_OT8 | GPIO_OTYPER_OT9);
    GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPD8_Msk | GPIO_PUPDR_PUPD9_Msk);
    GPIOB->OSPEEDR |= (3U << GPIO_OSPEEDR_OSPEED8_Pos)
        | (3U << GPIO_OSPEEDR_OSPEED9_Pos);

    //enable CAN1 clock
    RCC->APB1ENR1 |= RCC_APB1ENR1_CAN1EN;

    //exit sleep, enter init mode
    CAN1->MCR &= ~CAN_MCR_SLEEP;
    CAN1->MCR |= CAN_MCR_INRQ;
    while (!(CAN1->MSR & CAN_MSR_INAK))
        ;

    //bit-timing (500 kbit/s @ 80 MHz PCLK)
    CAN1->BTR = 0;
    CAN1->BTR |= (10U - 1U) << CAN_BTR_BRP_Pos;    //BRP = 10
    CAN1->BTR |= (1U - 1U) << CAN_BTR_SJW_Pos;      //SJW = 1
    CAN1->BTR |= (11U - 1U) << CAN_BTR_TS1_Pos;     //TS1 = 11
    CAN1->BTR |= (4U - 1U) << CAN_BTR_TS2_Pos;     //TS2 = 4

    // auto bus-off recovery & wake-up
    CAN1->MCR |= CAN_MCR_ABOM | CAN_MCR_AWUM;
    CAN1->MCR &= ~CAN_MCR_NART;    //allow retransmit
    CAN1->MCR &= ~CAN_MCR_INRQ;    //leave init

    //wait

```

```

while (CAN1->MSR & CAN_MSR_INAK)
    ;

//filter 0 -> accept all into FIFO0
CAN1->FMR |= CAN_FMR_FINIT;
CAN1->FA1R &= ~(1U << 0);
CAN1->FS1R |= (1U << 0);    //32-bit
CAN1->FM1R &= ~(1U << 0);    //mask mode
CAN1->sFilterRegister[0].FR1 = 0x00000000;
CAN1->sFilterRegister[0].FR2 = 0x00000000;
CAN1->FFA1R &= ~(1U << 0);    //FIFO0
CAN1->FA1R |= (1U << 0);    //enable
CAN1->FMR &= ~CAN_FMR_FINIT;

//interrupts
CAN1->IER |= CAN_IER_FMPIE0 | CAN_IER_ERRIE | CAN_IER_BOFIE |
CAN_IER_EPVIE | CAN_IER_EWGIE | CAN_IER_LECIE;
NVIC_SetPriority(CAN1_RX0_IRQn, 3);
NVIC_EnableIRQ(CAN1_RX0_IRQn);
NVIC_SetPriority(CAN1_SCE_IRQn, 4);
NVIC_EnableIRQ(CAN1_SCE_IRQn);
}

/*-----
 * function : CAN1_SendFrame()
 * INs      : id (uint32_t), dlc (uint8_t), *data (uint8_t)
 * OUTs     : none
 * author   : Bryan Lew (BL) - bglew@calpoly.edu
 * action   : load first free Ix mailbox; poll until RQCP or timeout
 * version  : 1
 * date     : 06/05/25
 *-----*/
void CAN1_SendFrame(uint32_t id, uint8_t dlc, const uint8_t *data) {
    uint32_t mb;
    volatile uint32_t tmo = 0;

    //pick a free mailbox
    if (CAN1->TSR & CAN_TSR_TME0)
        mb = 0;
    else if (CAN1->TSR & CAN_TSR_TME1)
        mb = 1;
    else if (CAN1->TSR & CAN_TSR_TME2)
        mb = 2;
    else
        return; //no room

    //identifier & DLC
    CAN1->sTxMailBox[mb].TIR = id << CAN_TI0R_STID_Pos;
    CAN1->sTxMailBox[mb].TDTR = dlc & CAN_TDT0R_DLC_Msk;

    //payload little-endian into TDLR/TDHR
    uint32_t tdlr = 0, tdhr = 0;
    if (dlc > 0)
        tdlr |= data[0];
    if (dlc > 1)

```

```

        tdlr |= data[1] << 8;
    if (dlc > 2)
        tdlr |= data[2] << 16;
    if (dlc > 3)
        tdlr |= data[3] << 24;
    if (dlc > 4)
        tdhr |= data[4];
    if (dlc > 5)
        tdhr |= data[5] << 8;
    if (dlc > 6)
        tdhr |= data[6] << 16;
    if (dlc > 7)
        tdhr |= data[7] << 24;
    CAN1->sTxMailBox[mb].TDLR = tdlr;
    CAN1->sTxMailBox[mb].TDHR = tdhr;

    //kick transmit
    CAN1->sTxMailBox[mb].TIR |= CAN_TI0R_TXRQ;

    //wait
    while (!(CAN1->TSR & (CAN_TSR_RQCP0 << mb)) && (tmo++ < 0xFFFFF))
        ;
}

/*-----
 * function : CAN1_RX0_IRQHandler()
 * author   : Bryan Lew (BL) - bglew@calpoly.edu
 * action    : On FIFO0 msg → extract & process; clear overrun if set.
 * version   : 1
 * date      : 06/05/25
 *-----*/
void CAN1_RX0_IRQHandler(void) {
    //FIFO0 message pending
    if (CAN1->RF0R & CAN_RF0R_FMP0_Msk) {
        uint32_t rir = CAN1->sFIFOMailBox[0].RIR;
        uint32_t rdtr = CAN1->sFIFOMailBox[0].RDTR;
        uint32_t rdldr = CAN1->sFIFOMailBox[0].RDLR;
        uint32_t rdhr = CAN1->sFIFOMailBox[0].RDHR;

        uint32_t id = (rir & CAN_RI0R_STID_Msk) >> CAN_RI0R_STID_Pos;
        uint8_t dlc = rdtr & CAN_RDT0R_DLC_Msk;

        uint8_t data[8] = { 0 };
        if (dlc > 0)
            data[0] = rdldr & 0xFF;
        if (dlc > 1)
            data[1] = (rdldr >> 8) & 0xFF;
        if (dlc > 2)
            data[2] = (rdldr >> 16) & 0xFF;
        if (dlc > 3)
            data[3] = (rdldr >> 24) & 0xFF;
        if (dlc > 4)
            data[4] = rdhr & 0xFF;
        if (dlc > 5)
            data[5] = (rdhr >> 8) & 0xFF;

```

```

        if (dlc > 6)
            data[6] = (rdhr >> 16) & 0xFF;
        if (dlc > 7)
            data[7] = (rdhr >> 24) & 0xFF;

        CAN1->RF0R |= CAN_RF0R_RFOM0; //release FIFO
        Process_OBD_Request(id, dlc, data);
    }

    //FIFO0 overrun
    if (CAN1->RF0R & CAN_RF0R_FOVR0)
        CAN1->RF0R |= CAN_RF0R_FOVR0;
}

/*-----*/
* function : Process_OBD_Request()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
* action    : Respond to 0x7DF functional mode-01 PID query (RPM, temp, speed)
* version   : 1
* date      : 06/05/25
*-----*/
void Process_OBD_Request(uint32_t id, uint8_t dlc, const uint8_t *d) {
    if (id != OBD_REQUEST_ID)
        return;
    if (dlc != 8 || d[0] != 0x02 || d[1] != 0x01)
        return;

    uint8_t pid = d[2];
    uint8_t rsp[8] = { 0 };
    uint8_t rsp_dlc = 0;

    rsp[1] = 0x41; //reply to mode 0x01
    rsp[2] = pid;

    switch (pid) {

        //handle engine rpm PID request
        case PID_ENGINE_RPM:
            uint16_t raw = current_rpm * 4; //get raw by multiplying by 4
            rsp[0] = 4;
            rsp[3] = raw >> 8;
            rsp[4] = raw & 0xFF;
            rsp_dlc = 5;
            break;

            //handle coolant temp PID request
        case PID_COOLANT_TEMP:
            rsp[0] = 3;
            rsp[3] = (uint8_t) (current_coolant_temp + 40);
            rsp_dlc = 4;
            break;

            //handle vehicle speed PID request
        case PID_VEHICLE_SPEED:
            rsp[0] = 3;

```

```

        rsp[3] = current_speed_kph;
        rsp_dlc = 4;
        break;

        //handle unsupported PID request
    default:
        return;
}

CAN1_SendFrame(OBD_RESPONSE_ID, rsp_dlc, rsp); //send CAN frame
}

/*-----*/
* function : CAN1_SCE_IRQHandler()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
* action    : Clear error flags so bus keeps running.
* version   : 1
* date      : 06/05/25
*-----*/
void CAN1_SCE_IRQHandler(void) {
    if ((CAN1->ESR & CAN_ESR_LEC_Msk)
        && (((CAN1->ESR & CAN_ESR_LEC_Msk) >> CAN_ESR_LEC_Pos) != 0)
        && (((CAN1->ESR & CAN_ESR_LEC_Msk) >> CAN_ESR_LEC_Pos) != 7)) {
        CAN1->ESR &= ~CAN_ESR_LEC_Msk;    //reset the previous error code
    }
}

/*-----*/
* function : SystemClock_Config()
* author   : Bryan Lew (BL) - bglew@calpoly.edu
* action    : MSI → PLL → 80 MHz SYSCLK
* authors   : STM32 Authors
* version   : 1
* date      : 06/05/25
*-----*/
void SystemClock_Config(void) {
    //power / voltage scaling
    RCC->APB1ENR1 |= RCC_APB1ENR1_PWREN;
    PWR->CR1 &= ~PWR_CR1_VOS_1;
    PWR->CR1 |= PWR_CR1_VOS_0;
    while (PWR->SR2 & PWR_SR2_VOSF)
        ;

    //enable MSI (4 MHz default)
    RCC->CR |= RCC_CR_MSION;

    //wait
    while (!(RCC->CR & RCC_CR_MSIRDY))
        ;

    //PLL = 4 MHz × 40 / 2 = 80 MHz
    RCC->PLLCFGR = 0;
    RCC->PLLCFGR |= RCC_PLLCFGR_PLLSRC_MSI;
    RCC->PLLCFGR |= 40U << RCC_PLLCFGR_PLLN_Pos; //N = 40
    RCC->PLLCFGR |= RCC_PLLCFGR_PLLREN; //R output on

```

```

RCC->CR |= RCC_CR_PLLON;

//wait
while (!(RCC->CR & RCC_CR_PLLRDY))
    ;

//flash latency / caches
FLASH->ACR = FLASH_ACR_LATENCY_4WS |
FLASH_ACR_PRFTEN | FLASH_ACR_ICEN | FLASH_ACR_DCEN;

//switch SYSCLK -> PLLR
RCC->CFGR |= RCC_CFGR_SW_PLL;

//wait
while ((RCC->CFGR & RCC_CFGR_SWS_Msk) != RCC_CFGR_SWS_PLL)
    ;

//AHB & APB prescalers = DIV1
RCC->CFGR |= RCC_CFGR_HPRE_DIV1 |
RCC_CFGR_PPRE1_DIV1 | RCC_CFGR_PPRE2_DIV1;

SystemCoreClockUpdate();
}

```